

ΑΝΑΛΥΤΙΚΗ ΔΕΔΟΜΕΝΩΝ

Απαλλακτική Εργασία (ατομική)

Όνοματεπώνυμο : Αλέξιος Καστανάρας
ΑΜ : Π22062
email : alexioskast@gmail.com

Εισαγωγή.....	2
Βήμα 1 : Προπαρασκευή δεδομένων (Data preprocessing).....	3
Βήμα 2 : Συσταδοποίηση (Clustering).....	5
KMeans.....	5
DBSCAN.....	6
Σύγκριση αλγορίθμων K-Means και DBSCAN.....	8
Βήμα 3 : Ταξινόμηση (Classification / Regression).....	8
(a) Classification - Πρόβλεψη της κατηγορίας παχυσαρκίας (NObesity).....	8
Classifying using RandomForestClassifier.....	9
Classifying Using Neural Networks with early stopping.....	10
Classifying using RandomForestClassifier VS Classifying Using Neural Networks.....	12
(b) Regression - Πρόβλεψη της τιμής του Δείκτη Μάζας Σώματος (Body Mass Index - BMI).....	12
Σύνοψη.....	15

Εισαγωγή

Η παρούσα εργασία πραγματοποιήθηκε στο πλαίσιο του μαθήματος «Αναλυτική Δεδομένων» και έχει ως στόχο την ανάλυση του συνόλου δεδομένων “*Estimation of Obesity Levels Based On Eating Habits and Physical Condition*”. Το σύνολο περιλαμβάνει πληροφορίες για 2.111 άτομα και αφορά συνήθειες διατροφής, φυσική δραστηριότητα και σωματικά χαρακτηριστικά. Σκοπός είναι η πρόβλεψη του δείκτη μάζας σώματος (BMI) και της κατηγορίας παχυσαρκίας.

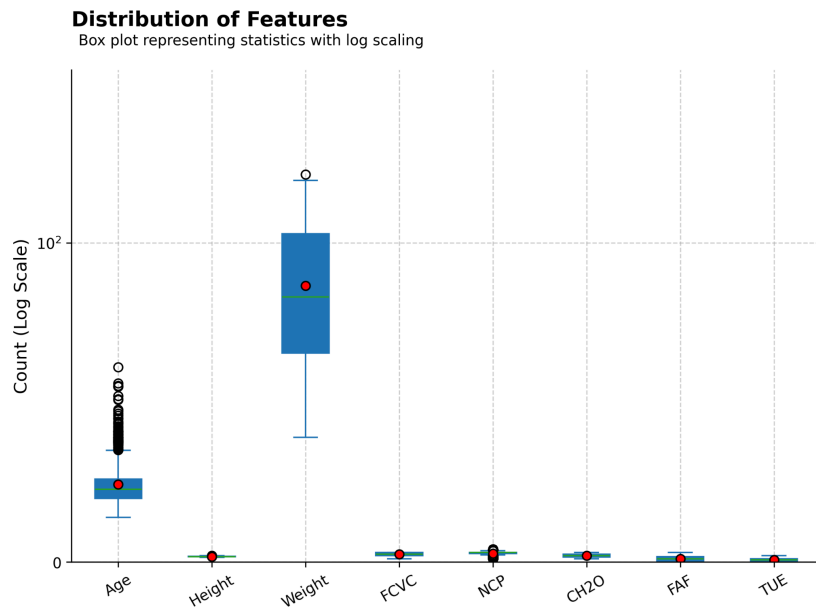
Η εργασία αυτή υλοποιήθηκε σε ένα ενιαίο αρχείο Python (ergasia.py) και περιλαμβάνει τα εξής βήματα:

1. **Προεπεξεργασία Δεδομένων:** Καθαρισμός και προετοιμασία των δεδομένων για την ανάλυση, που περιλαμβάνει την κανονικοποίηση, την κωδικοποίηση και τη μείωση των διαστάσεων.
2. **Συσταδοποίηση (Clustering):** Χρήση τεχνικών συσταδοποίησης (K-means, DBSCAN) για την ανακάλυψη προτύπων και ιδιοτήτων του συνόλου δεδομένων.
3. **Ταξινόμηση και Παλινδρόμηση (Classification & Regression):** Εφαρμογή αλγορίθμων ταξινόμησης για την πρόβλεψη της κατηγορίας παχυσαρκίας και αλγορίθμων παλινδρόμησης για την πρόβλεψη του δείκτη BMI.
4. **Σύνοψη και Εξαγωγή Συμπερασμάτων:** Παρουσίαση των αποτελεσμάτων και ανάλυση των παρατηρήσεων που προέκυψαν από τα προηγούμενα βήματα.

Στην αρχή του αρχείου Python πραγματοποιούνται τα απαραίτητα import όλων των βιβλιοθηκών που χρησιμοποιούνται καθολικά για την υλοποίηση αυτών των βημάτων. Οι βασικές βιβλιοθήκες που χρησιμοποιήθηκαν καταγράφονται στο αρχείο resources.txt και περιλαμβάνουν τις εξής: matplotlib, missingno, numpy, pandas, seaborn, scikit-learn, kneed, keras, tensorflow.

Βήμα 1 : Προπαρασκευή δεδομένων (Data preprocessing)

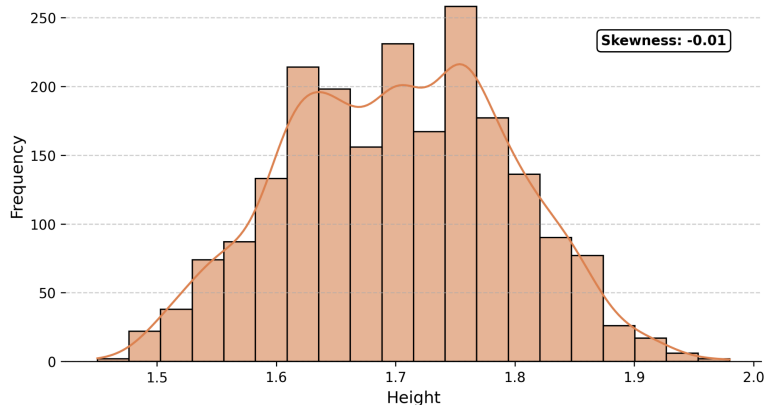
α) Για αρχή, εκτυπώνω το `head()` και το `info()` του dataset μου αφού το έχω διαβάσει, για να δω αν υπάρχουν ελλειπίες ή/και εσφαλμένες τιμές. Στη συνέχεια, εκτυπώνω το `describe()` από όλα τα αριθμητικά χαρακτηριστικά, για να δω το εύρος των τιμών κάθε στήλης. Δημιουργώ όλα τα Distribution plots των `numeric_feats` ξεχωριστά και ένα `fig` που έχει όλα τα Density Plots μαζί και τέλος ένα box plot του dataset.



Στο dataset δεν υπάρχουν τιμές τύπου null ούτε λανθασμένες, και οι ακραίες τιμές των στηλών είναι εντάξει. Ξεχωρίζουν όμως οι στήλες του ύψους και του βάρους, οι οποίες παρόλα αυτά είναι φυσιολογικές δηλαδή κάποιιοι έχουν όντως ψηλό ύψος ή βάρος. Από τη συνάρτηση `'describe()'` βλέπουμε τις ελάχιστες και μέγιστες τιμές αυτών των δύο και διαπιστώνουμε ότι είναι όντως φυσιολογικές, οπότε, σε αυτό το στάδιο δεν αλλάζω κάτι στο dataset.

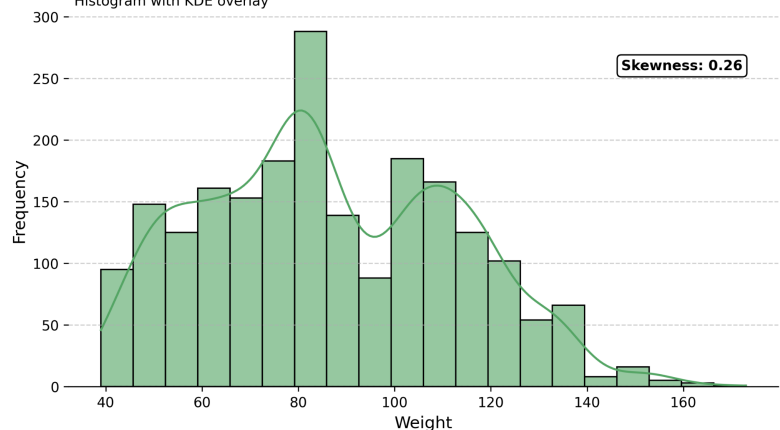
Height Distribution

Histogram with KDE overlay



Weight Distribution

Histogram with KDE overlay



β) Στρογγυλοποιώ τις τιμές του πεδίου "FCVC" και στη συνέχεια, με χρήση του MinMaxScaler, θέτω την ελάχιστη τιμή το 0 και την μέγιστη το 1. Τοποθετώ όλες τις ενδιαμέσες τιμές των αριθμητικών χαρακτηριστικών μέσα σε αυτό το εύρος.

```
#reduced the number of different values
data["FCVC"] = data["FCVC"].round(2)

#normalize all numeric_feats
scaler = MinMaxScaler()
data[numeric_feats] = scaler.fit_transform(data[numeric_feats])
```

γ) Με τη χρήση της .map() αντιστοιχώ, στις στήλες που έχουν μόνο δύο κατηγορίες όπως MALE/FEMALE και YES/NO, τις τιμές 1 και 0 αντίστοιχα. Στη συνέχεια, επιλέγω ποιες είναι οι στήλες που είναι κατηγορικές (Categorical) και εφαρμόζω LabelEncoder σε καθεμία ξεχωριστά.

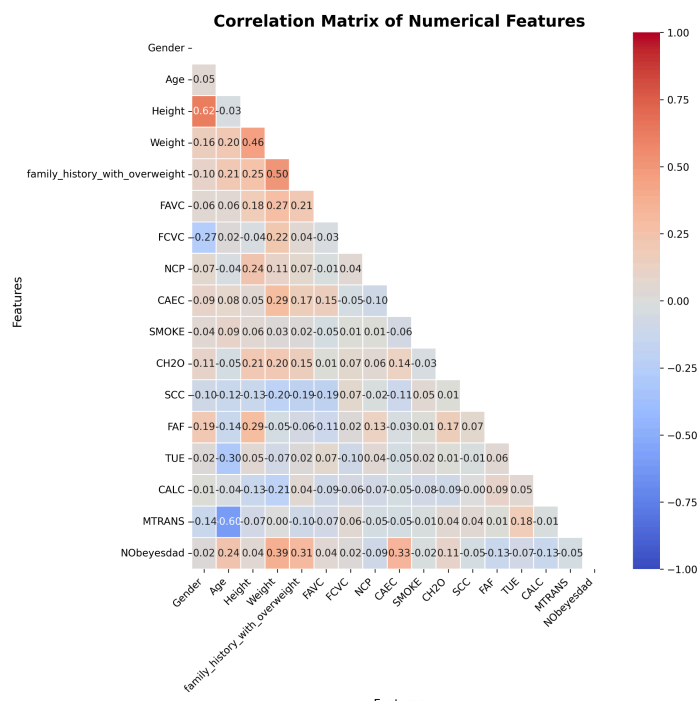
```
#normalize binary options to 1/0
data["Gender"] = data["Gender"].map({"Male": 1, "Female": 0})
data["family_history_with_overweight"] = data["family_history_with_overweight"].map({"yes": 1, "no": 0})
data["FAVC"] = data["FAVC"].map({"yes": 1, "no": 0})
data["SMOKE"] = data["SMOKE"].map({"yes": 1, "no": 0})
data["SCC"] = data["SCC"].map({"yes": 1, "no": 0})

#normalize using LabelEncoder for Categorical feats
columns_to_encode = ["CAEC", "CALC", "MTRANS", "NObeyesdad"]

encoder = LabelEncoder()
for column in columns_to_encode:
    data[column] = encoder.fit_transform(data[column])
```

δ) Εξετάζω τη συσχέτιση μεταξύ των χαρακτηριστικών, ώστε αν υπάρχουν ζεύγη με υψηλό βαθμό συσχέτισης, να κρατήσω μόνο το ένα από αυτά. Με αυτόν τον τρόπο να μειωθεί ο όγκος των δεδομένων και να αποφεύγεται η πολυδιάστατη πλεοναστικότητα.

```
corr = data.select_dtypes(include=["number"]).corr(method="pearson").round(2)
```



Το correlation δεν είναι αρκετό του για να αφαιρέσουμε κάποια χαρακτηριστικά.

Βήμα 2 : Συσταδοποίηση (Clustering)

KMeans

Φτιάχνω ένα array με τα χαρακτηριστικά που σχετίζονται με τη διατροφή και κάνω αντίγραφο από το dataset μόνο αυτά. Χρησιμοποιώ PCA κρατώντας όσες συνιστώσες χρειάζονται λιγότερες σε αριθμό ώστε να μπορούν να διατηρήσουν το 95% της πληροφορίας και στη συνέχεια βρίσκω και το k, που είναι ο αριθμός των διαφορετικών τιμών που έχει το χαρακτηριστικό NObeyesdad. Τέλος, εφαρμόζω clustering με αυτό το k και random_state = 42.

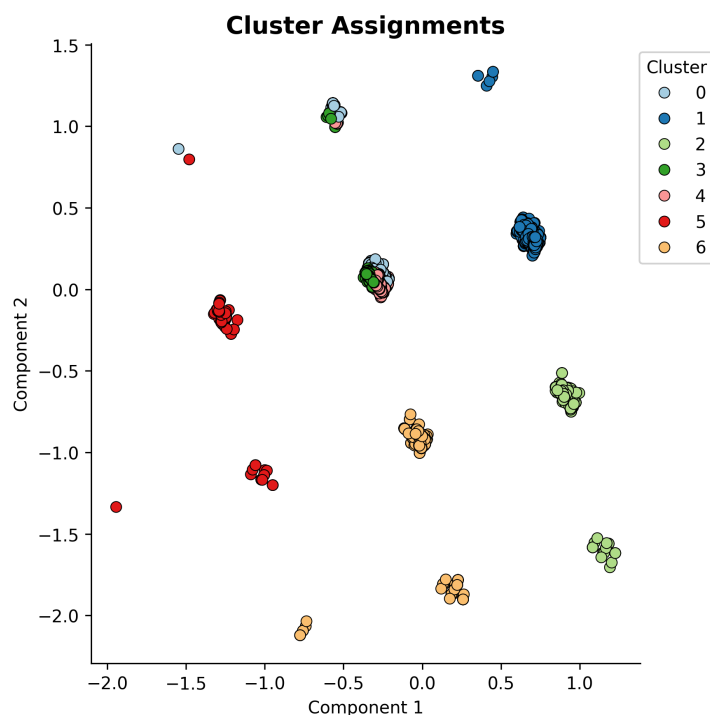
```
features = [
    "FCVC", #Do you eat high caloric food frequently?
    "NCP", #How many main meals do you have daily?
    "CAEC", #Do you eat any food between meals?
    "CH2O", #How much water do you drink daily?
    "CALC", #How often do you drink alcohol?
]

new_data = data[features].copy()

pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(new_data)

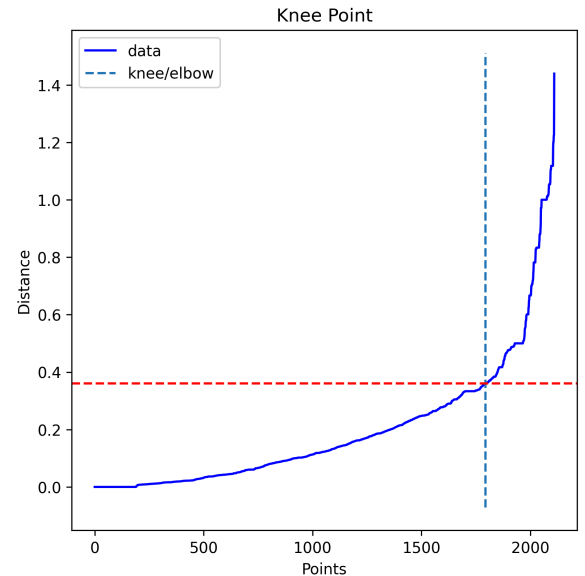
k = data["NObeyesdad"].nunique()

start_time = datetime.now()
kmeans = KMeans(n_clusters=k, random_state=42)
clusters = kmeans.fit_predict(X_pca)
stop_time = datetime.now()
print("KMeans time :"+ str(stop_time-start_time))
silhouette = silhouette_score(X_pca, clusters)
print("KMeans Silhouette Score:", silhouette)
```



DBSCAN

Πριν κάνω clustering με τη μέθοδο DBSCAN, ορίζω τον αριθμό γειτόνων `number\_of\_neighbors=20` (αυτόν τον αριθμό τον είδα να λειτουργεί καλύτερα ώστε ο αριθμός κλάσεων να είναι κοντά στον σωστό αριθμό) και στη συνέχεια, με τη συνάρτηση `NearestNeighbors()` αναζητώ τους γείτονες κάθε εγγραφής στο `X\_pca` (το dataset όπως προέκυψε μετά το PCA). Έπειτα κρατάω τις αποστάσεις αυτών των γειτόνων και, με τη χρήση της βιβλιοθήκης `KneeLocator`, βρίσκω τη μέγιστη απόσταση που χρειάζεται ώστε ένα στοιχείο να θεωρηθεί γειτονικό. Τέλος, τρέχω τον αλγόριθμο DBSCAN με τις παραμέτρους (eps=distances[knee.knee], n_jobs=-1)

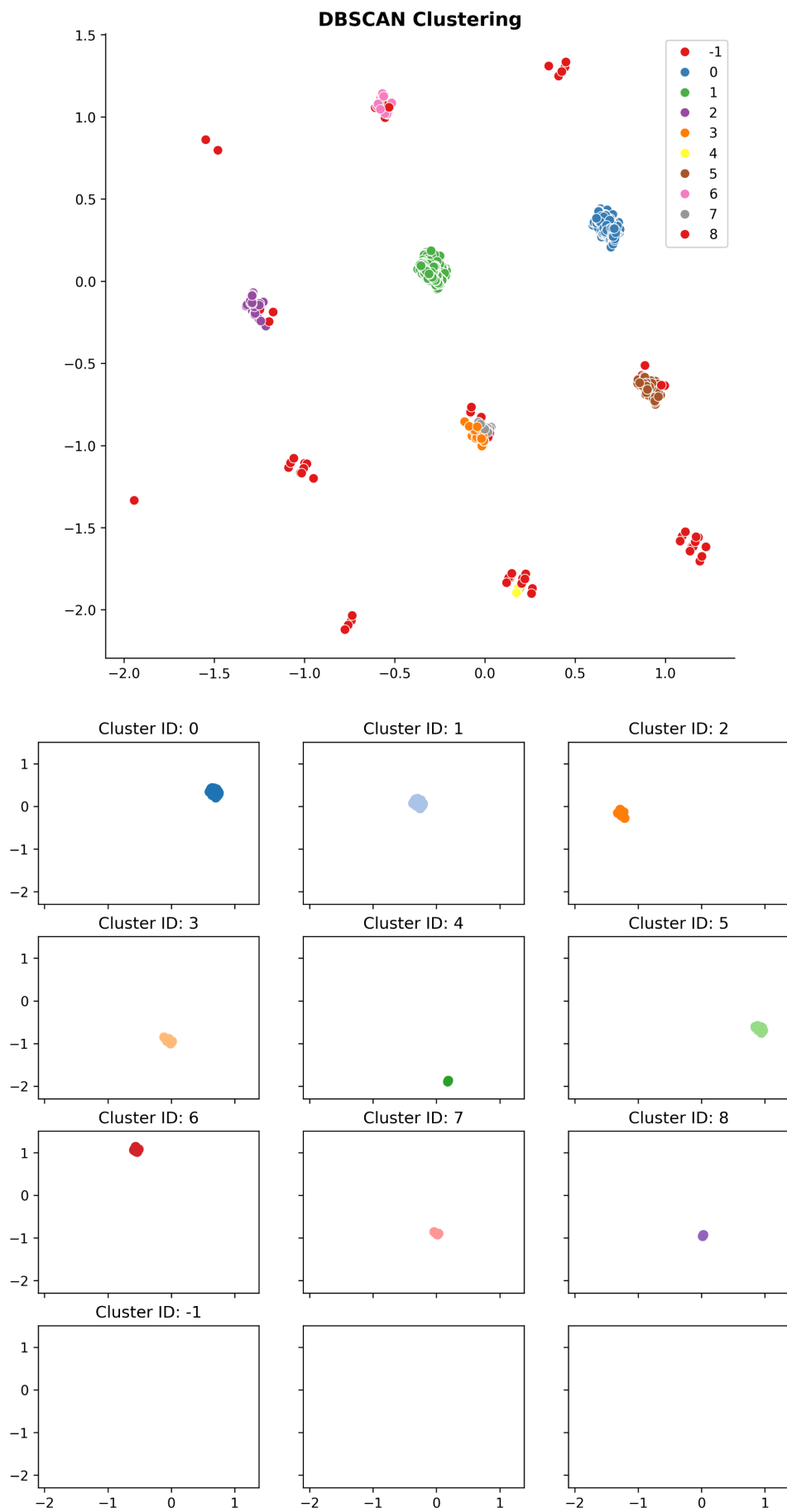


```
#find maximum distances
number_of_neighbors = 20
nearest_neighbors = NearestNeighbors(n_neighbors = number_of_neighbors)
neighbors = nearest_neighbors.fit(X_pca)

distances, indices = neighbors.kneighbors(X_pca)
distances = np.sort(distances[:, number_of_neighbors - 1], axis=0)

start_time = datetime.now()
dbscan = DBSCAN(eps=distances[knee.knee], n_jobs=-1)
dbscan.fit(X_pca)
stop_time = datetime.now()
print("DBSCAN time :"+ str(stop_time-start_time))

labels = dbscan.labels_
silhouette = silhouette_score(X_pca, labels)
print("DBSCAN Silhouette Score:", silhouette)
```



Σύγκριση αλγορίθμων K-Means και DBSCAN

Από πλευράς απόδοσης χρησιμοποιώντας το Silhouette Score, ο K-Means έχει σκορ **0.358** έναντι **0.445** του DBSCAN (περίπου **24.3% διαφορά**). Από πλευράς χρόνου εκπαίδευσης, ο K-Means χρειάζεται **0.140205** δευτερόλεπτα, ενώ ο DBSCAN **0.031255** δευτερόλεπτα (ο K-Means είναι περίπου **3.49 φορές πιο αργός**). Από πλευράς ευκολίας υλοποίησης του κώδικα, ο K-Means είναι πιο απλός σε σχέση με τον DBSCAN.

Βήμα 3 : Ταξινόμηση (Classification / Regression)

(a) Classification - Πρόβλεψη της κατηγορίας παχυσαρκίας (NObesity)

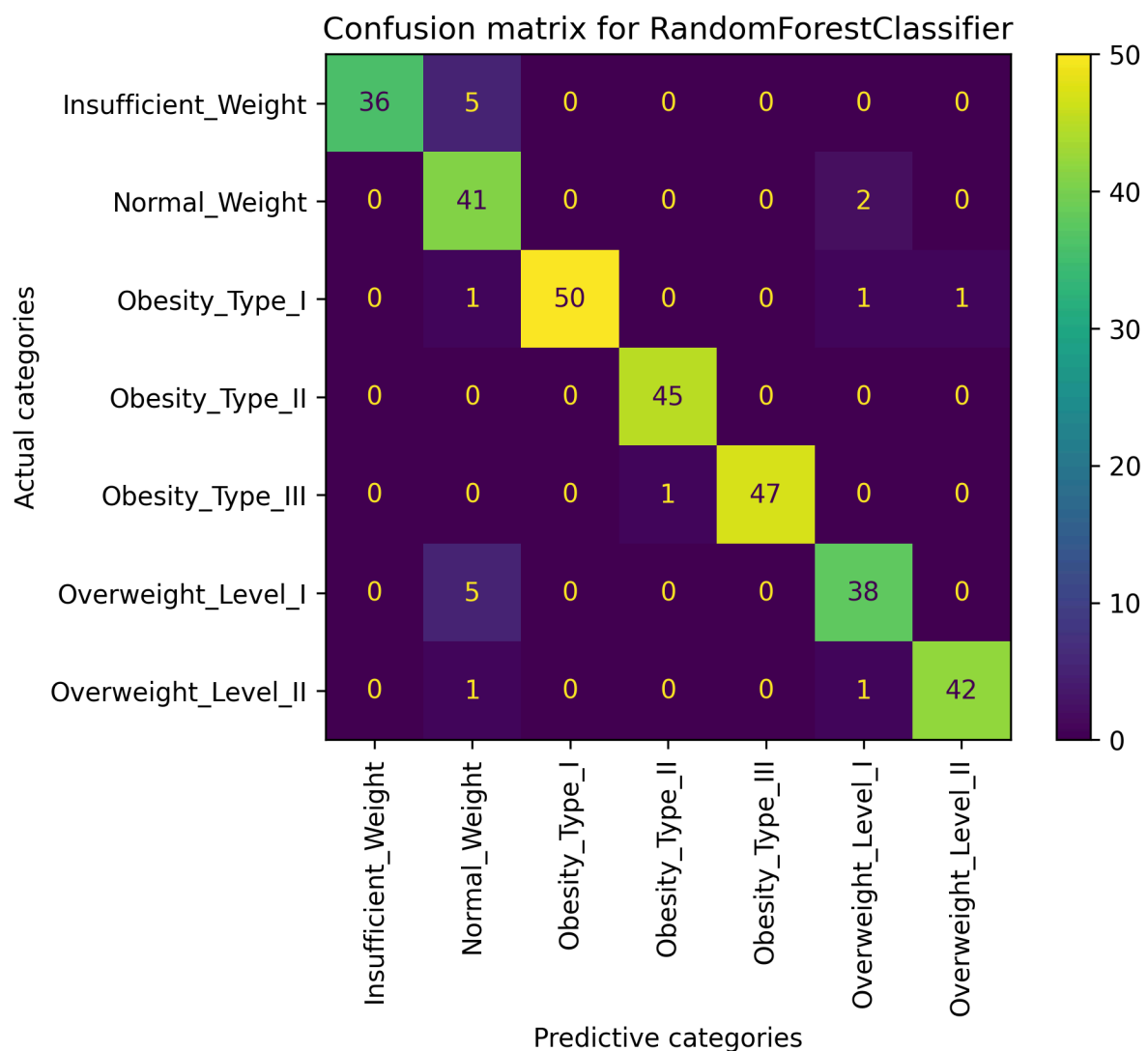
Κάνω set στο X όλο το dataset εκτός από τη στήλη "NObesidad", και στο target y τη στήλη "NObesidad". Ορίζω ότι θα σπάσω το dataset σε 70%-20%-10% για train, dev και test αντίστοιχα. Στη συνέχεια, με τη συνάρτηση **train_test_split**, χωρίζω πρώτα το dataset σε train και dev+test, και μετά ξαναχρησιμοποιώ το **train_test_split** για να χωρίσω το dev+test σε dev και test. Και στις δύο περιπτώσεις χρησιμοποιώ το stratify, ώστε να διατηρούνται ίσες αναλογίες κλάσεων κάθε φορά.



Classifying using RandomForestClassifier

Χρησιμοποιώ τον RandomForestClassifier και θέτω να δημιουργήσει 500 δέντρα, χωρίς όριο στο βάθος τους, χρησιμοποιώντας το "entropy" ως κριτήριο και random_state = 42 .

```
rf = RandomForestClassifier(  
    n_estimators = 500,  
    criterion = "entropy",  
    max_depth = None,  
    random_state = 42,  
)  
rf.fit(X.iloc[ix_train], y.iloc[ix_train])
```



Classifying Using Neural Networks with early stopping

Παίρνω τον FC_MNIST από την Keras και φτιάχνω ένα δικό μου νευρωνικό δίκτυο, το οποίο έχει ως είσοδο shape (16,) και αποτελείται από 3 επίπεδα (layers) με αριθμό νευρώνων αντίστοιχα 128-16-7. Τα πρώτα 3 επίπεδα έχουν activation="relu", ενώ το τελευταίο επίπεδο έχει activation="softmax". Για να λειτουργήσει σωστά, κάνω πρώτα one-hot encode τη μεταβλητή y, και στη συνέχεια το κάνω train χρησιμοποιώντας το callback EarlyStopping, θέτοντας το **patience=20**.

Κάνω model.compile με: optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"] και ορίζω **epochs=200**.

```
class FC_MNIST(models.Model):
    def __init__(self):
        super(FC_MNIST, self).__init__()
        self.fc1 = layers.Dense(units=128, activation="relu") # Hidden layer
        self.fc2 = layers.Dense(units=16, activation="relu") # Hidden layer
        self.fc3 = layers.Dense(units=7, activation="softmax") # Output layer

    def call(self, input_tensor):
        x = self.fc1(input_tensor)
        x = self.fc2(x)
        return self.fc3(x)

# Create the input layer
input_layer = layers.Input(shape=(16,))

# Instantiate the custom model and use it on the input layer
model = FC_MNIST()(input_layer)

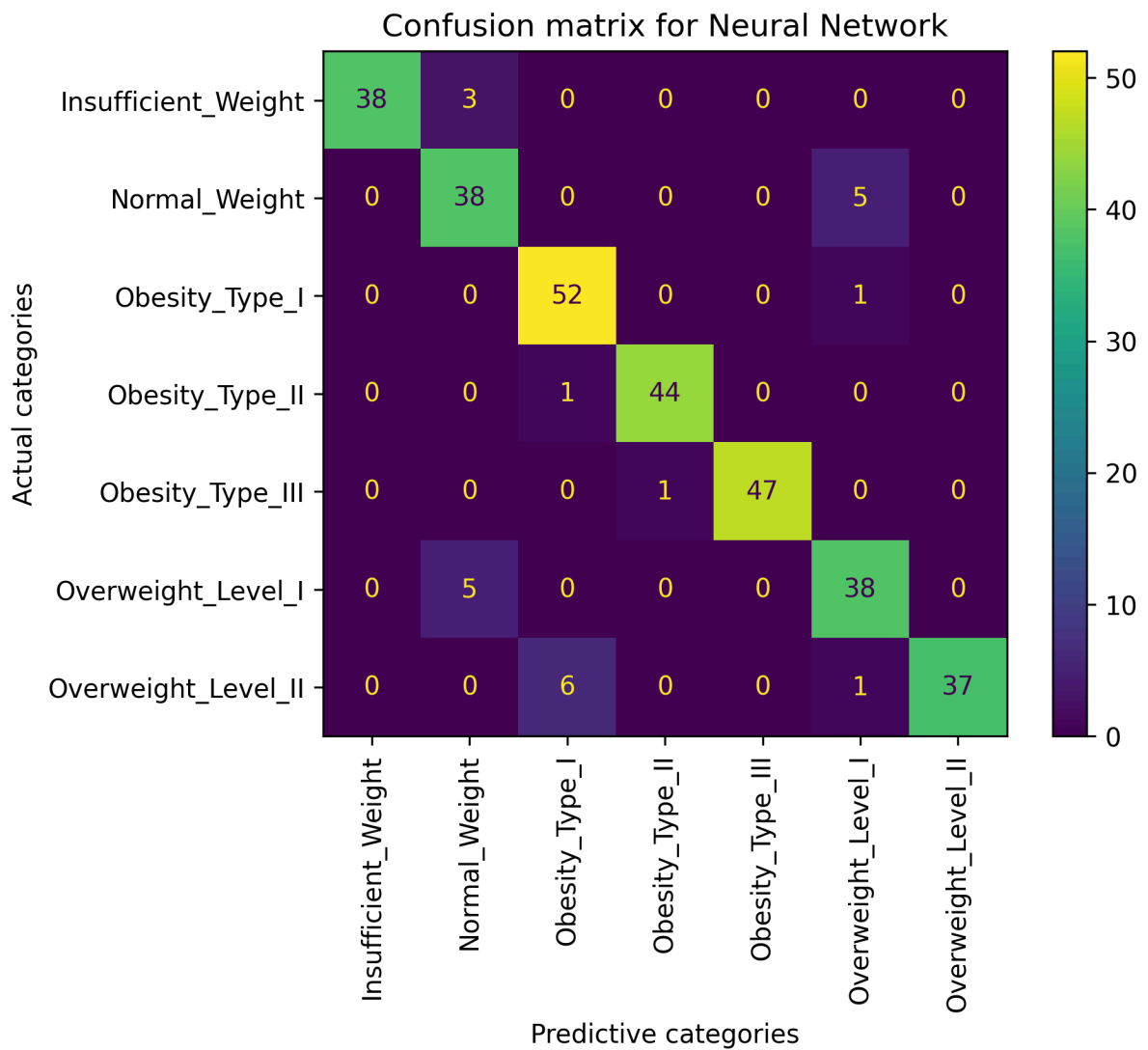
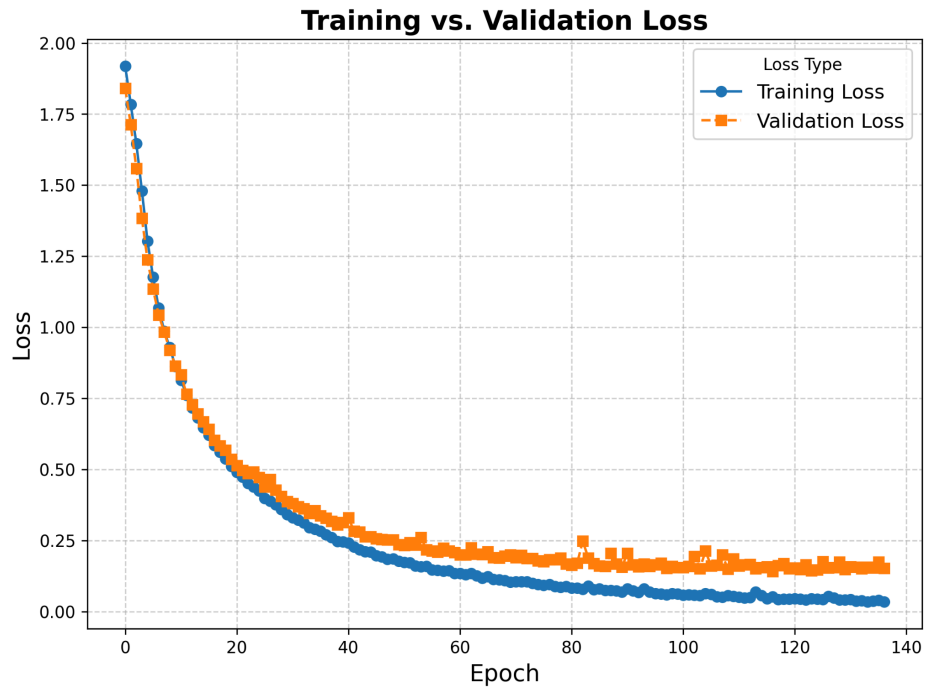
# Create a complete Keras model by specifying the inputs and outputs
model = models.Model(inputs=input_layer, outputs=model)

#encode y
y_onehot = pd.get_dummies(pd.Series(y[ix_train])).values
y_dev_onehot = pd.get_dummies(pd.Series(y[ix_dev])).values

callback = callbacks.EarlyStopping(monitor="val_loss", patience=20)
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])

model_hist = model.fit(
    X.iloc[ix_train],
    y_onehot,
    epochs=200,
    validation_data=(X.iloc[ix_dev], y_dev_onehot),
    callbacks=[callback],
)

model_hist
```



Classifying using RandomForestClassifier VS Classifying Using Neural Networks

Για να συγκρίνω την απόδοση ανάμεσα στον RandomForestClassifier και στο Νευρωνικό Δίκτυο, χρησιμοποιώ το weighted F1 score. Αυτό το σκορ δείχνει πόσο καλά ταξινομεί κάθε μοντέλο, λαμβάνοντας υπόψη την απόδοση σε όλες τις κλάσεις με βάση τη συχνότητά τους.

```
score = f1_score(
    y[ix_test],
    model.predict(X.iloc[ix_test]).argmax(axis=1),
    average="weighted",
    labels=np.arange(k),
)

print(f"Weighted F1 score: {score:.3f}")
```

Το F1 score του RandomForestClassifier είναι **0.945** και του Νευρωνικού Δικτύου είναι **0.934**

(b) Regression - Πρόβλεψη της τιμής του Δείκτη Μάζας Σώματος (Body Mass Index - BMI)

Ξαναφορτώνω το dataset από το αρχείο CSV και προσθέτω μια νέα στήλη με όνομα "BMI", στην οποία υπολογίζω τον Δείκτη Μάζας Σώματος (BMI) για κάθε εγγραφή. Στη συνέχεια, κάνω κανονικοποίηση του dataset όπως στο βήμα 1.

Έπειτα, χρησιμοποιώ το FC_MNIST της βιβλιοθήκης Keras για να δημιουργήσω ένα δικό μου νευρωνικό δίκτυο, το οποίο έχει είσοδο διαστάσεων (17,) και αποτελείται από τέσσερα επίπεδα με 64, 32, 8 και 1 units αντίστοιχα. Τα τρία πρώτα επίπεδα χρησιμοποιούν ενεργοποίηση ReLU, ενώ το τελευταίο επίπεδο χρησιμοποιεί γραμμική ενεργοποίηση (linear), καθώς πρόκειται για πρόβλημα παλινδρόμησης.

Για την εκπαίδευση του μοντέλου χρησιμοποιώ το callback EarlyStopping με **patience=20**, και το μοντέλο το κάνω compile με βελτιστοποιητή Adam, συνάρτηση κόστους **mean squared error** και μέτρηση απόδοσης **mean absolute error**, ενώ ορίζω αριθμό **epochs=200**.

```

class FC_MNIST(models.Model):
    def __init__(self):
        super(FC_MNIST, self).__init__()
        # Creating layers in the initializer
        self.fc1 = layers.Dense(units=64 , activation="relu") # Hidden layer
        self.fc2 = layers.Dense(units=32, activation="relu") # Hidden layer
        self.fc3 = layers.Dense(units=8, activation="relu") # Hidden layer
        self.fc4 = layers.Dense(units=1, activation="linear") # Output layer

    def call(self, input_tensor):
        # Pass input_tensor through the layers sequentially
        x = self.fc1(input_tensor)
        x = self.fc2(x)
        x = self.fc3(x)
        return self.fc4(x)

# Create the input layer
input_layer = layers.Input(shape=(17,))

# Instantiate the custom model and use it on the input layer
model = FC_MNIST()(input_layer)

# Create a complete Keras model by specifying the inputs and outputs
model = models.Model(inputs=input_layer, outputs=model)

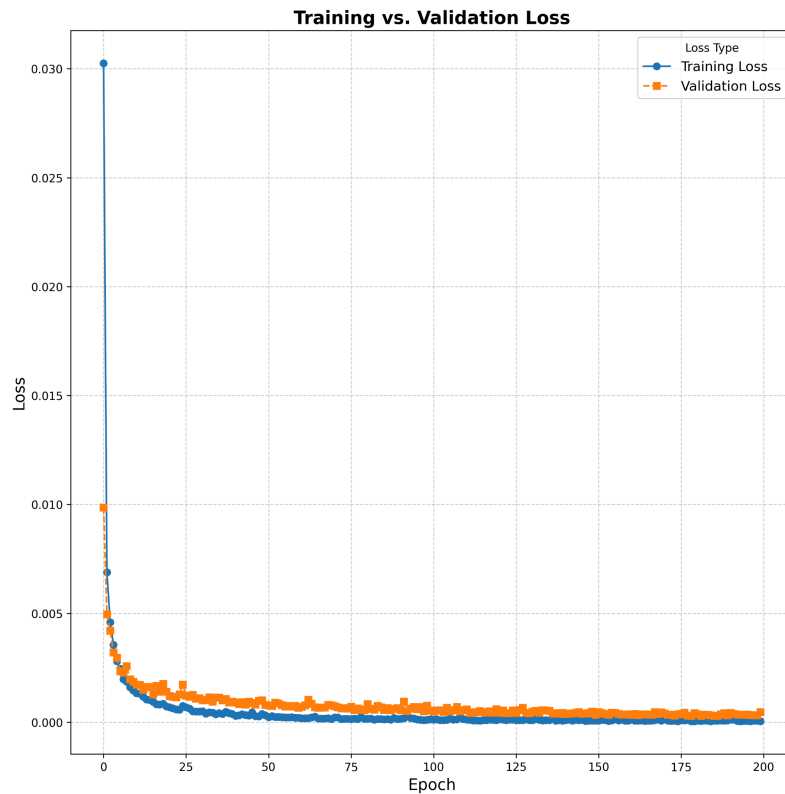
# Model summary
model.summary(expand_nested=True)

y_train = pd.Series(y[ix_train]).values
y_dev = pd.Series(y[ix_dev]).values

callback = callbacks.EarlyStopping(monitor="val_loss", patience=20)
model.compile(optimizer="adam", loss="mean_squared_error", metrics=["mean_absolute_error"])

model_hist = model.fit(
    X.iloc[ix_train],
    y_train,
    epochs=200,
    validation_data=(X.iloc[ix_dev], y_dev),
    callbacks=[callback],
)
model_hist

```



Αφού εκπαιδεύτηκε το μοντέλο, χρησιμοποίησα τα δεδομένα του test set για να κάνω προβλέψεις και υπολόγισα το MAPE (Mean Absolute Percentage Error), δηλαδή το μέσο ποσοστιαίο σφάλμα των προβλέψεων. Το MAPE δείχνει, σε ποσοστό %, πόσο μακριά βρίσκονται κατά μέσο όρο οι προβλέψεις από τις πραγματικές τιμές.

```
y_pred = model.predict(X.iloc[ix_test]).flatten()
mape = np.mean(np.abs((y.loc[ix_test] - y_pred) / y.loc[ix_test])) * 100

print(f"MAPE: {mape:.2f}%")
```

Οι τιμές BMI που προβλέπει το μοντέλο μου απέχουν κατά μέσο όρο περίπου **3,82%** από τις πραγματικές τιμές BMI.

Σύνοψη

Στην παρούσα εργασία μελέτησα δεδομένα που αφορούν διατροφικές συνήθειες και φυσική κατάσταση, με σκοπό να κατανοήσω πώς σχετίζονται με τα επίπεδα παχυσαρκίας. Αφού καθάρισα και προετοίμασα τα δεδομένα, εφάρμοσα τεχνικές μηχανικής μάθησης για ανάλυση και πρόβλεψη.

Στο στάδιο της ομαδοποίησης, ο αλγόριθμος DBSCAN παρουσίασε καλύτερη απόδοση από τον K-Means, σχηματίζοντας πιο καθορισμένες ομάδες και με μικρότερο υπολογιστικό κόστος.

Στην ταξινόμηση των επιπέδων παχυσαρκίας, τόσο το μοντέλο Random Forest όσο και το Νευρωνικό Δίκτυο απέδωσαν εξαιρετικά, με το Random Forest να υπερέχει ελαφρώς σε ακρίβεια.

Τέλος, η πρόβλεψη του Δείκτη Μάζας Σώματος (BMI) μέσω του Νευρωνικού Δικτύου είχε πολύ μικρό σφάλμα, δείχνοντας πως το μοντέλο μπορεί να προσφέρει αξιόπιστα αποτελέσματα.